

DataMining LINFO2275: Gestures Recognition

Group 9

Berlaimont Gaetan (DATE)

Henrion Benoit (MAP)

Zeamari Yassine (DATS)

May 18, 2025

1 Introduction

In this project, we used two datasets: domain 1, which contains digit gestures, and domain 4, which consists of 3D shapes. To classify the 3D shapes in Domain 4, we created a mapping from each shape to a corresponding digit label.

We implemented two approaches: a baseline method using Dynamic Time Warping (DTW) with k-nearest neighbors, and a deep learning model.

Both methods were applied to each dataset and evaluated using user-independent and user-dependent cross-validation to assess their classification performance.

2 Pre-processing steps

As pre-processing step we choose to remove the mean and scale to unit variance for each trajectories. This avoid shift between trajectories, so that we can easily see the number in the dataset. Then we performed a PCA.

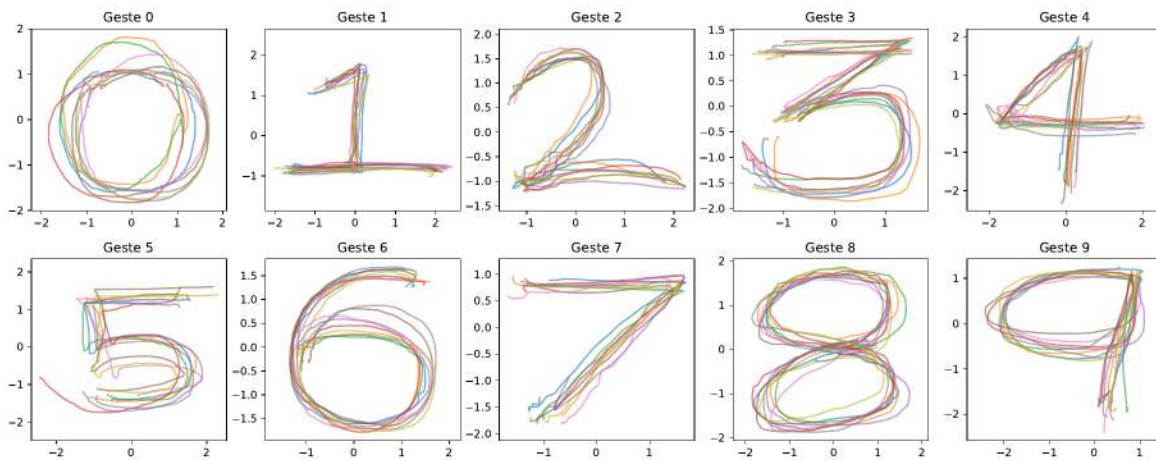


Figure 1: samples before doing PCA

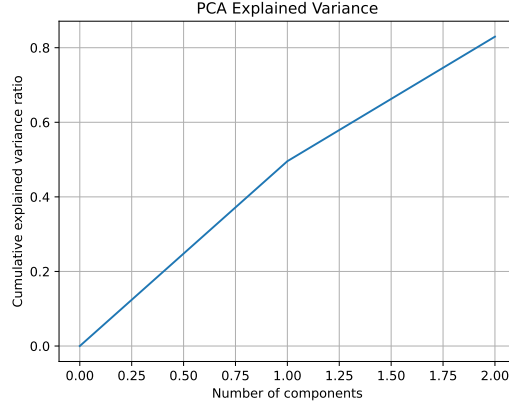


Figure 2: explain variance ration

As we can see in the graph above we only used the 2 best component for simplicity since the total variance explained is above 80%.

3 Baseline methods theoretical explanation

3.1 k-Nearest Neighbors

The k-Nearest Neighbors (K-NN) algorithm is a non-parametric supervised learning method used for classification. The algorithm predicts its label $\hat{y}$ by identifying the k training samples that are closest to x , based on a distance metric—typically the Euclidean distance:

$$d(x, y) = \sqrt{\sum_{j=1}^d (x_j - y_j)^2}$$

The predicted class is determined by majority voting among the k nearest neighbors:

$$\hat{y} = \arg \max_{c \in C} \sum_{i \in N_k(x)} w_i \cdot \mathbb{I}[y_i = c]$$

each neighbor contributes to the vote with a weight inversely proportional to the square of the distance:

$$w_i = \frac{1}{d(x, y)^2}$$

3.2 Dynamic time wrapping

Dynamic Time Warping (DTW) is a distance measure designed to compare temporal sequences that may vary in speed or length. Unlike Euclidean distance, DTW finds an optimal alignment (warping path) between two sequences by stretching or compressing them in time.

Let $X = (x_1, x_2, \dots, x_n)$ and $Y = (y_1, y_2, \dots, y_m)$ be two sequences of possibly different lengths. DTW computes the distance $\text{DTW}(X, Y)$ using a dynamic programming approach:

$$\text{DTW}(i, j) = \|x_i - y_j\| + \min \begin{cases} \text{DTW}(i-1, j) \\ \text{DTW}(i, j-1) \\ \text{DTW}(i-1, j-1) \end{cases}$$

with the boundary conditions:

$$\text{DTW}(0, 0) = 0, \quad \text{DTW}(i, 0) = \text{DTW}(0, j) = \infty \text{ for } i, j > 0$$

4 Implementation & methodology

4.1 Baseline method

Algorithm 1 Dynamic Time Warping (DTW)

```
1: Initialize matrix  $\text{DTW}[0 \dots n, 0 \dots m] \leftarrow \infty$ 
2:  $\text{DTW}[0, 0] \leftarrow 0$ 
3:  $w \leftarrow \max(w, |n - m|)$ 
4: for  $i = 1$  to  $n + 1$  do
5:   for  $j = \max(1, i - w)$  to  $\min(m, i + w + 1)$  do
6:     Compute Euclidean cost between  $s_1[i - 1]$  and  $s_2[j - 1]$ 
7:      $\text{DTW}[i, j] \leftarrow \text{cost} + \min \begin{pmatrix} \text{DTW}[i - 1, j], \\ \text{DTW}[i, j - 1], \\ \text{DTW}[i - 1, j - 1] \end{pmatrix}$ 
8:   end for
9: end for
10: return  $\text{DTW}[-1, -1]$ 
    =0
```

Algorithm 2 k-NN classification using DTW

```
1: for each training sample  $(\mathbf{x}_i, y_i)$  do
2:    $d_i \leftarrow \text{DTW}(\mathbf{x}_i, \mathbf{x}_{\text{test}}, w)$ 
3:   Store pair  $(d_i, y_i)$ 
4: end for
5: Sort all  $(d_i, y_i)$  by  $d_i$  in ascending order
6: Select the  $k$  labels associated with the smallest distances
7:  $\hat{y} \leftarrow$  majority label among the  $k$  nearest neighbors
8: return  $\hat{y}$ 
    =0
```

4.2 Deep learning method

For the state-of-the-art method we chose to use a deep network structure with the package *tensorflow*. The structure of the network is the following :

- input layer of size 2×85 (see below)
- hidden layer of size 128 with relu activation.
- dropout layer with $p=0.3$ to avoid overfitting.
- hidden layer of size 64 with relu activation.
- output layer of size 10 with softmax activation.

One big concern for this task is the fact that deep learning use fixed size input which is not the case for the trajectories in this dataset. To fix this problem we choose to resize all trajectories by interpolating them to a constant size (85 which is the median of all trajectories size) instead of padding zero or cutting end (This approach preserves "temporal alignment" and avoids information loss). Since we have two coordinate we have an input size of 170. The output of the network directly represent the class of the sample.

5 Validation and comparison tests

In this section, we evaluate and compare the results obtained on both datasets using two methods: Dynamic Time Warping with k-nearest neighbors, and a deep learning model. Each method is assessed with user-dependent and user-independent cross-validation to analyze their generalization performance. Additionally, we use confusion matrices to gain deeper insight into the classification results. In the following results, we used $k = 3$ for the k-nearest neighbors algorithm.

5.1 first dataset (domain 1)

Fold	DTW with K-NN		Deep Learning Model	
	User-Dep.	User-Indep.	User-Dep.	User-Indep.
Fold 1	1.00	0.98	1.00	0.98
Fold 2	0.99	0.90	1.00	0.92
Fold 3	0.99	1.00	1.00	0.98
Fold 4	1.00	0.99	1.00	0.99
Fold 5	1.00	0.88	1.00	0.99
Fold 6	1.00	1.00	1.00	0.99
Fold 7	1.00	1.00	1.00	1.00
Fold 8	1.00	1.00	1.00	0.98
Fold 9	1.00	1.00	1.00	1.00
Fold 10	1.00	0.93	1.00	1.00
Mean Accuracy	0.9770	0.9680	1.00	0.9930
Std. Deviation	0.0119	0.0442	0.0000	0.0100

Table 1: Comparison of DTW with K-NN and Deep Learning Model for User-Dependent and User-Independent Cross-Validation on Dataset 1 (digits gestures)

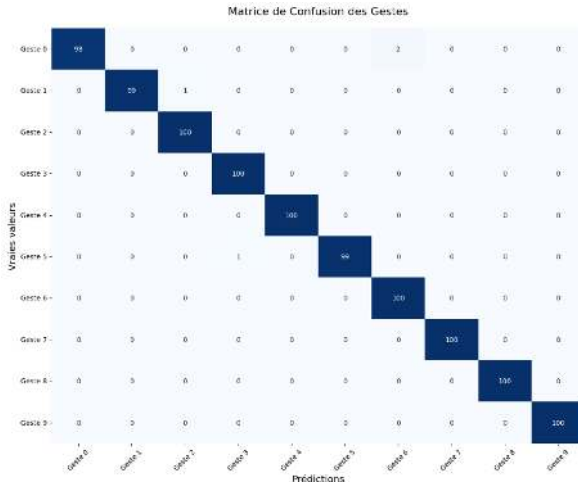


Figure 3: DTW with K-NN (User-Dep.)

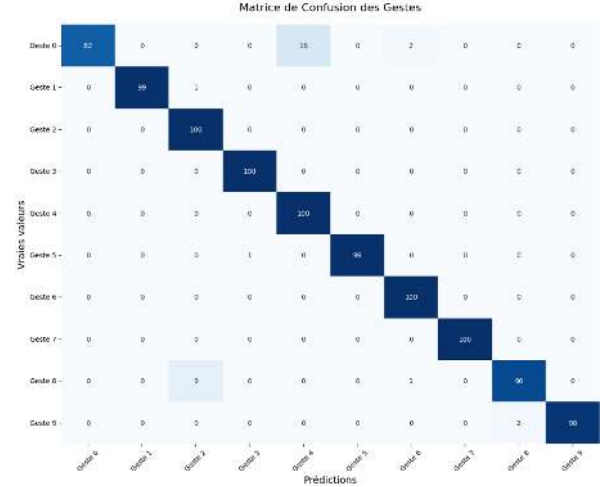


Figure 4: DTW with K-NN (User-Indep.)

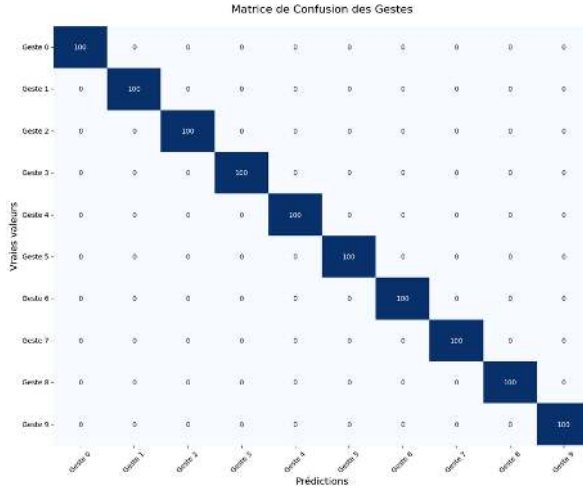


Figure 5: Deep Learning Model (User-Dep.)

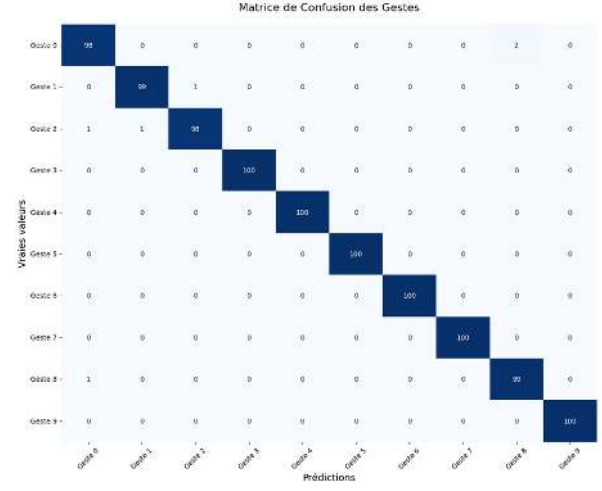


Figure 6: Deep Learning Model (User-Indep.)

5.2 second dataset (domain 4)

Fold	DTW with K-NN		Deep Learning Model	
	User-Dep.	User-Indep.	User-Dep.	User-Indep.
Fold 1	0.97	0.94	0.99	0.94
Fold 2	0.97	0.79	0.99	0.90
Fold 3	0.99	0.92	1.00	0.99
Fold 4	1.00	0.96	1.00	1.00
Fold 5	0.98	0.88	1.00	0.98
Fold 6	1.00	0.80	1.00	0.99
Fold 7	0.98	0.95	0.99	1.00
Fold 8	1.00	0.92	1.00	0.99
Fold 9	0.98	0.98	1.00	1.00
Fold 10	1.00	0.72	1.00	1.00
Mean Accuracy	0.9870	0.8860	0.9970	0.9790
Std. Deviation	0.0119	0.0824	0.0046	0.0314

Table 2: Comparison of DTW with K-NN and Deep Learning Model for User-Dependent and User-Independent Cross-Validation on Dataset 2 (3D Shapes)

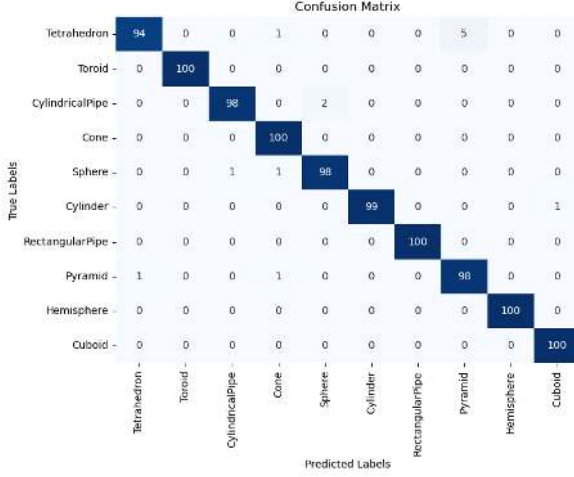


Figure 7: DTW with K-NN (User-Dep.)

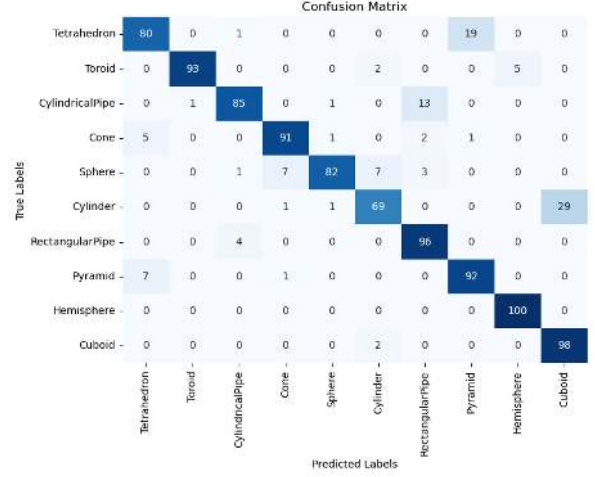


Figure 8: DTW with K-NN (User-Indep.)

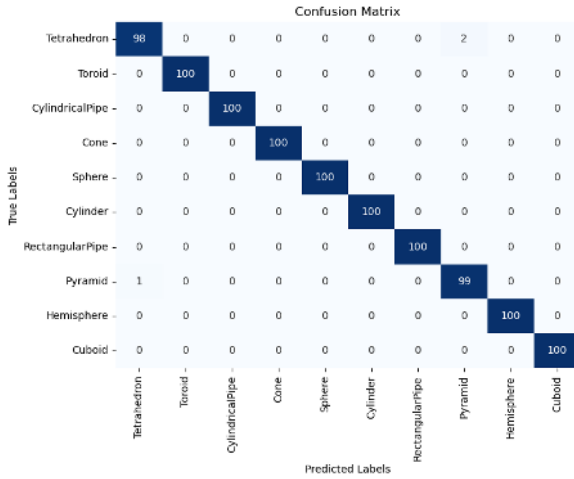


Figure 9: Deep Learning Model (User-Dep.)

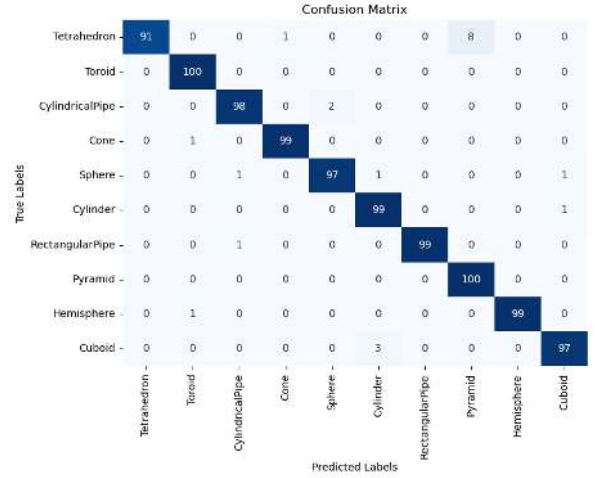


Figure 10: Deep Learning Model (User-Indep.)

5.3 Comments about the results

As expected, for both datasets and methods, user-dependent cross-validation showed higher mean accuracy than user-independent cross-validation. This happens because, in the user-dependent setup, the model is trained and tested on data from the same users, making the task easier. On the other hand, user-independent evaluation is harder since the model has to generalize to completely new users. For example, in dataset 1, fold 6 has a much lower accuracy of only 80%, which is probably because user 6 writes numbers in a way that is quite different from the other users. This also explains why the standard deviation is higher in user-independent cross-validation compared to user-dependent.

Overall, the deep learning model achieved higher mean accuracy than the DTW with k-NN method. The confusion matrices reveal some interesting patterns. For dataset 1 (digit gestures), DTW with k-NN often confused digit 8 with 2 (9 times), and digit 0 with 4 in the user-independent case (18 times). The deep model made fewer mistakes, with digit 0 misclassified as 8 twice (user-independent) and twice as 6 (user-dependent).

In dataset 2 (3D shapes), DTW with k-NN misclassified Tetrahedron as Pyramid 5 times in the user-dependent setting, and Cylinder as Cuboid 29 times in the user-independent setting. The deep model also confused Tetrahedron with Pyramid, but only 2 times (user-dependent) and 8 times (user-independent).

These results show that, on both datasets, our deep model generalizes better than the baseline method, which uses dynamic time warping combined with k-nearest neighbor.

6 Conclusion

This report compared a time warping method and a deep learning approach for classification tasks on two different datasets: one for numerical input and another for shape recognition. The evaluation showed that while both methods achieved nearly comparable accuracy in user-dependent scenarios, the deep learning model outperformed the time warping approach in user-independent settings. These results suggest that deep learning offers better generalization across users.